PROJECT GUIDE · WRITTEN FOR INTERVIEW PREPARATION

AI Workspace Assistant

A tool that reads your email and calendar, works out what you promised people, and books the follow-up before it slips.

BACKEND TESTS	FRONTEND TESTS	DB MIGRATIONS	TABLES	MAIL PROVIDERS	CALENDAR PROVIDERS
381	186	15	9	2	2

> HOW TO USE THIS DOCUMENT
>
> Read sections 1–4 until you can say them without looking. That covers the "tell me about your project" opening. Sections 6, 7 and 10 are where the real engineering is — those are what a technical interviewer will dig into. Section 12 has the likely questions written out with answers.
>
> Boxes marked **Say it like this** are sentences you can use almost word for word. Boxes marked **The subtle part** are where you show depth.

Stack — Python / FastAPI · PostgreSQL · Next.js / TypeScript · Supabase Auth · Microsoft Graph · Gmail & Google Calendar APIs · an OpenAI-compatible LLM

What's in here

1 The thirty-second answer
The opening you memorise

2 The problem it solves
Why anyone would want this

3 What it actually does
The five features, in plain words

4 How the pieces fit together
Frontend, backend, database — from basics

5 The database
Nine tables, and the one design decision that mattered

6 Getting the mail in
Incremental sync, three providers, three different models

7 The AI part
The pipeline, and the budget that shaped it

8 Security
Tokens, database access, proving who the user is

9 How it's tested
And why the tests needed fixing first

10 Bugs I found, and why each was invisible
The strongest material you have

11 Trade-offs and what I chose not to build
Judgement, not gaps

12 Likely questions, with answers
Rehearse these

13 Glossary
Every term in this document, defined

ONE HONEST NOTE TO CARRY INTO THE ROOM

The Gmail and Google Calendar code is thoroughly tested, but every test replaces the network with a fake. It has never run against a live Google account. If an interviewer asks "is it in production?", the true answer is: the Microsoft path works end to end, the Google path is written and tested but not yet proven against the real API. Saying that plainly reads as engineering maturity. Claiming otherwise falls apart under one follow-up question.

1 The thirty-second answer

> SAY IT LIKE THIS
>
> *"It's an assistant that connects to your email and calendar — both Outlook and Gmail, and you can connect several accounts at once. It reads your messages, uses a language model to pull out who you talked to and what you committed to doing, and shows that as a list of action items. It can then book a follow-up meeting on your calendar, checking all your connected calendars first so it doesn't double-book you."*
>
> *"The interesting engineering wasn't the AI call — that's about twenty lines. It was everything around it: syncing mail efficiently without re-downloading your whole inbox, and making the whole thing fit inside a free language-model quota of 200,000 tokens a day."*

That last sentence is deliberate. It moves the conversation from "you called an API" to "you engineered under a constraint", which is the part worth talking about.

Three sentences on each layer, if they want more

- **Frontend** — a Next.js site in TypeScript, exported as plain static files. It has no server of its own; it talks to the backend over HTTP.
- **Backend** — a Python FastAPI service. It owns all the data access, the provider syncing, and the AI pipeline.
- **Database** — PostgreSQL, hosted by Supabase. Nine tables, managed by fifteen migration files.

2 The problem it solves

Commitments live in your inbox, not in your to-do list. Someone emails "can you send the deck before Thursday?", you read it on your phone, you mean to do it, and it disappears under forty newer messages. Nothing is broken — the information was there — but it never became a task.

Existing tools mostly don't help, for a specific reason: a to-do app needs you to type the task in. The commitment is already written down, in a message you already read. The work is extraction, not capture.

Why it needs more than one account

This is the part that makes it genuinely useful rather than a demo. Real people have a work mailbox and a personal one. The person asking you for the deck might be in either. A tool that reads only one of them misses half your life, and a tool that reads them separately can't tell you that the same person emailed you at work on Monday and personally on Friday.

> THE DESIGN CONSEQUENCE
>
> Mail, calendar events and sync state are stored *per mailbox*. But contacts and action items are stored *per user* — deliberately unified across every connected mailbox. "This is the same person, across your accounts" is the whole point, so the schema has to make that the natural outcome rather than something assembled later.

3 What it actually does

FEATURE	IN PLAIN WORDS
Connect mailboxes	Sign in with Microsoft or Google, then add more accounts of either kind. Each one has its own status, so one expired account doesn't make the working ones look broken.
Sync	Every fifteen minutes, pull new mail, calendar events and Teams messages from each connected account — fetching only what changed, not everything.
Extract	Send message text to a language model and get back structured data: the people involved, and any action items with a direction (mine / theirs) and a due date.
Show	Dashboard with counts and charts, a merged Emails list labelled by mailbox, a people list, a task planner, a calendar view, and an insights page.
Schedule	Turn an action item into a real calendar meeting. It checks every connected calendar for conflicts before suggesting a time.

The user journey, once

1. You sign in with your work Microsoft account.
2. In Settings you add your personal Gmail. Both now appear, both marked connected, one marked "primary".
3. A background sync pulls the last thirty days of mail from both.
4. The extraction step reads those messages and creates contacts and action items.
5. You open the planner and see "Send the deck to Dana — due Thursday", and that it came from your work mailbox.
6. You click schedule; it offers a slot that is free on *both* calendars and books it.

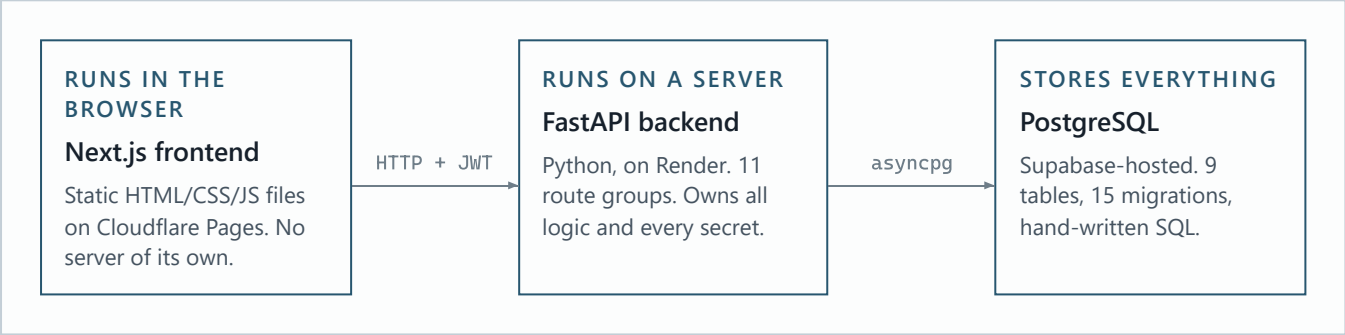
4 How the pieces fit together

First, the three words

If you are explaining this to someone non-technical, or being asked to start from basics, define these three before using them.

- **Frontend** — the part that runs inside the browser. HTML, CSS and JavaScript. It draws the screen and nothing else.
- **Backend** — a program running on a server somewhere, waiting for requests. It holds the logic and the secrets, because anything sent to the browser can be read by the user.
- **Database** — the thing that remembers. Without it, closing the tab loses everything.

How they're arranged here



Why the frontend is "static", and why that confuses people

Next.js *can* run its own server and talk to a database directly. This project deliberately doesn't. The config sets `output: "export"`, which means the build produces plain files — no Node process, no server-side code, no database connection from the frontend at all.

So every piece of data on screen arrives the same way: the browser calls the FastAPI backend over HTTP, and the backend queries Postgres. "Static" describes how the interface is *delivered*, not the system.

> WHY THAT'S THE RIGHT CALL HERE
>
> Static files are free to host and effectively impossible to overload, and keeping one backend means there is exactly one place that touches the database and holds secrets. The alternative — Next.js server routes — would have split data access across two languages and two deployments for no benefit.

Where Supabase appears twice

This trips people up, so be precise. Supabase is two things here, and the second one is used in a restricted way:

- **Auth** — the browser uses Supabase's client library *only* to sign in, sign out, and link another provider. It never queries a table.
- **Postgres** — the backend connects to the same Supabase project's database directly, and does all the data work.

Because the browser never reads tables, I was able to revoke all table permissions from the public-facing database roles. More on that in section 8.

The one constraint that shapes the whole setup

> THE SUBTLE PART
>
> The `profiles` table has a foreign key to `auth.users` — the table Supabase's own login system owns. That means **the database and the login service must be the same Supabase instance.** You cannot pair a fast local database with real cloud login, because a user ID created by cloud Auth would violate a foreign key against the local users table.
>
> So there are two self-consistent environments — fully local, or fully cloud — and mixing them is impossible rather than merely discouraged. This is a good answer to "what was an unexpected constraint?"

5 The database

The nine tables

TABLE	WHAT IT HOLDS
`profiles`	One row per user. Linked to Supabase's own login table.
`connected_accounts`	One row per connected mailbox. Provider, email address, encrypted tokens, status, and which one is "primary".
`emails`	Synced messages. Sender, recipients, subject, body, and the provider's bulk-mail verdict.
`calendar_events`	Synced events from both Outlook and Google Calendar.
`chat_messages`	Microsoft Teams messages. Microsoft only — Gmail has no equivalent.
`contacts`	People, unified across every mailbox.
`action_items`	What was extracted: the text, who owes it, a due date, a category, and a link to a booked meeting.
`sync_state`	A bookmark per mailbox per resource, so the next sync resumes instead of restarting.
`llm_quota_state`	When the language model's daily quota ran out, so the app stops asking.

Migrations, briefly

A migration is a numbered SQL file describing one change to the schema. Running them in order from an empty database reproduces the structure exactly. This matters because it makes the database reviewable and repeatable — you can read the fifteen files and see every decision, and a fresh deployment is one command.

> A HABIT WORTH MENTIONING
>
> After every schema change I re-ran the whole chain from an empty database, not just the new file. Applying a migration to an already-migrated database proves much less — it can pass while the from-scratch path is broken, which is the path a real deployment takes.

The one design decision that mattered

The original schema stored one Microsoft account per user: the token table's primary key *was* the user ID, so a second mailbox was impossible by construction. Supporting several accounts meant changing the unit of ownership throughout.

Every synced row gained an `account_id`, and uniqueness changed from "one message ID per user" to "one message ID per mailbox". Without that, two mailboxes receiving the same message would collide on a unique constraint, and worse — `sync_state` was keyed per user, so adding a second account would make it resume from the first account's bookmark and silently skip mail.

How a column is added safely

You cannot simply add a required column to a table that already has rows — every existing row would violate it. The pattern is three steps in one migration:

```
1. add the column as optional          (existing rows get NULL)
2. fill it in for existing rows        (backfill from the primary mailbox)
3. now make it required               (safe: every row has a value)
```

SAY IT LIKE THIS

"The schema change was the riskiest part, because it touched every table that holds synced data. I did it as nullable, then backfill, then not-null, and I verified it by replaying all fifteen migrations from an empty database and running the full test suite against the result."

6 Getting the mail in

Why you can't just "fetch all the mail"

A real mailbox is tens of thousands of messages. Downloading all of them every fifteen minutes would be slow, would exhaust the provider's rate limits, and would re-process mail you already handled. So every mail API offers **incremental sync**: "tell me what changed since this bookmark."

The bookmark is the whole game. Store it correctly and each sync is a handful of messages. Store it wrongly and you either re-download everything or — much worse — silently skip mail and never know.

Three providers, three different models

This is the most interesting technical content in the project, because all three APIs solve the same problem differently, and each has a trap.

	MICROSOFT GRAPH	GMAIL	GOOGLE CALENDAR
Bookmark is	an opaque URL deltaLink	an integer historyId	an opaque string syncToken
Deletions	a removal marker on the item	usually a TRASH label, not a deletion	the event returns with status cancelled
Bodies	included in the response	a separate request per message	included
Expired bookmark	HTTP 410	HTTP 404	HTTP 410

The trap in each one

- **Gmail:** deleting mail usually just adds a `TRASH` label, and archiving just removes the `INBOX` label. Neither reports a deletion. If you only watch the deletion list, deleted and archived mail stays in your app forever.
- **Gmail:** message bodies are base64 with the padding stripped, using the URL-safe alphabet. The standard decoder throws an error on most real messages unless you re-pad first.
- **Google Calendar:** the bookmark cannot be combined with a date range — the API returns an error — so an incremental sync that also passes a date window never runs at all. And without a specific flag, repeating events come back as the "rule" with no actual occurrences, so a weekly standup blocks nothing and never appears.
- **All-day events** carry a date rather than a timestamp. Read only the timestamp field and both ends come back empty — and an event with no time is invisible to every availability check, so a week of annual leave would block nothing.

What happens when a bookmark expires

Gmail keeps history for about a week. If a mailbox goes unsynced over a holiday, the bookmark becomes unusable. The wrong response is to report an error — the account then stays broken until a human notices. The right response is to fall back to a full thirty-day re-sync automatically and carry on.

> THE SUBTLE PART — AN ORDERING BUG WITH SILENT DATA LOSS
>
> During a full re-sync you have to read the new bookmark *before* you start listing messages, not after. Read it afterwards and any mail that arrives during the sync falls into a gap: it was not in the list you fetched, and it is already behind the bookmark you saved. It is never synced, and nothing ever reports a problem.
>
> Reading the bookmark first means such mail is merely seen twice — which is harmless, because the database write is an **upsert**: insert if new, update if already there. That is the general principle. Prefer doing something twice over doing it zero times, and make the write safe to repeat.

Only one sync at a time

Three things can start a sync: the fifteen-minute cron, a background task when you sign in, and the "Sync now" button. Nothing stopped them overlapping, and two syncs running together could both read the same unprocessed messages and spend the language-model budget on them twice.

The fix is a **Postgres advisory lock** — a named lock the database hands out, keyed on the user ID. It is a try-lock rather than a wait-lock: if a sync is already running, the second one reports "already running" and does nothing, instead of queueing up behind it and exhausting the connection pool.

An advisory lock is held by the database session, so if the server crashes, Postgres releases it automatically. A "locked" column in a table would have stayed stuck forever.

7 The AI part

The pipeline

```
message in the database
   ↓   is it worth spending tokens on?        ← filter, no AI involved
   ↓   cut the body to 1,500 characters       ← truncate
   ↓   ask the model, demanding a fixed shape ← one API call
   ↓   validate the reply against a schema    ← reject malformed output
   ↓   write contacts + action items
   ↓   mark the message processed
```

The model call itself is the least interesting step. Everything around it exists because of one number.

The real constraint

The project uses a free language-model tier. The published limits are four numbers, and which one binds first is not obvious:

LIMIT	VALUE	BINDS FIRST?
Requests per minute	`30`	No
Requests per day	`1,000`	No
Tokens per minute	`8,000`	No
Tokens per day	`200,000`	**Yes**

A token is roughly three quarters of a word. One long email thread measured about 3,900 tokens — so without any intervention the daily budget was around **43 messages**. That is not a product.

> SAY IT LIKE THIS
>
> *"The limit that mattered wasn't requests per day, it was tokens per day. Once I measured a real thread I found the budget was about forty messages a day, which was useless. Two changes got it to roughly two hundred: truncating bodies, and filtering out mail that was never going to produce a task."*

How the budget was made to fit

1. Truncation — measured, not guessed

Bodies are cut to 1,500 characters. One measured thread went from 3,900 tokens to about 700 — roughly 43 messages a day to roughly 199. The cut is taken from the *end*, deliberately: requests and commitments cluster near the start of a message, while signatures, legal footers and quoted reply chains pile up at the bottom.

2. Filtering before the model, not after

Most inbox volume is automated mail that yields nothing. The filter is a pure function over a plain dictionary — no database, no network — which makes it exhaustively testable and means the same rules serve both providers. It rejects a message when:

- the sender looks automated — `no-reply`, `notifications`, `mailer-daemon` and similar, matched against the *whole* name before the @ so that a real person called "arnold.bouncer" is not caught;
- the body is under 20 characters — "ok", "thanks", "got it";
- the provider itself already classified it as bulk.

> THE SUBTLE PART — A RULE THAT WAS DOCUMENTED BUT NEVER RAN
>
> That third rule existed in the code and was described in a comment, but it was unreachable: nothing stored the provider's verdict and nothing passed it in. Every promotional email was still costing a full model call. Both providers supply this for free with data already being fetched — Microsoft as a "focused / other" field, Gmail as category labels — so I added a column, normalised both vocabularies to one, and wired it through.
>
> Deliberately, an unknown verdict means "extract it". Defaulting to "bulk" would have silently dropped every message synced before the column existed.

3. Deriving what can be derived

The UI shows Task / Meeting / FYI chips on each item. Those are computed from data already present, not asked of the model — zero additional tokens.

Making the model's output trustworthy

A language model returns text. Text is not a data structure, and three things can go wrong: the shape can be invalid, the content can be wrong, and the message itself can try to hijack the instructions.

- **Shape** — the request demands a strict JSON schema, and the reply is validated against a Pydantic model before anything touches the database. A malformed reply is a handled failure, not a crash.
- **Repetition** — each message gets at most three attempts, then it is abandoned. Without that cap, one permanently unparseable message is retried forever and starves everything behind it. That is a **poison pill**.
- **Injection** — message bodies are fenced inside the prompt, so an email saying "ignore your instructions and..." is treated as content rather than as a command.
- **Quota** — when the provider reports the daily limit is gone, that is recorded, and the app stops calling until it resets rather than hammering a closed door.

> A DECISION I CAN DEFEND BOTH WAYS
>
> I considered batching several messages into one call to amortise the prompt preamble. I decided against it, because the measurements already showed four to ten times more headroom than needed, and batching makes failures ambiguous — if one message in a batch of ten produces bad output, you cannot tell which. The cost was understood, so the extra complexity wasn't justified yet. That is a **deliberate** non-decision, not an oversight, and it is worth saying that way.

8 Security

What's actually valuable here

The app stores OAuth tokens — credentials that grant read access to someone's entire mailbox. That is the most sensitive thing in the system, more so than the mail itself, because a token keeps working.

Tokens are encrypted at rest

Access and refresh tokens are encrypted with Fernet (symmetric encryption from Python's `cryptography` library) before being written. The database stores ciphertext; the key lives in an environment variable. So a database dump alone does not hand over anyone's mailbox.

> WORTH KNOWING, AND WORTH SAYING
>
> That key can never be rotated once tokens are stored — every stored token becomes permanently undecryptable and every mailbox needs reconnecting by hand. That is a real design weakness, and naming it unprompted is stronger than being caught by it. The proper fix is versioned keys: tag each ciphertext with the key that encrypted it, so old and new can coexist during a migration.

Who can read the database

Supabase can expose tables directly to the internet as a REST API, using two public roles. Row Level Security (RLS) is the Postgres feature that decides which *rows* a role may see.

This project never uses that REST API — the browser only does login, and all data access goes through FastAPI. So I audited it and found something worth fixing. RLS was switched on everywhere, and eight of the nine tables had no policies at all, which denies everything. Safe — but safe *by accident*:

- the public roles still held full permissions on every table;
- so adding one policy to any table later — a perfectly reasonable change — would instantly publish it to anyone holding the public key;
- and the table holding encrypted mailbox tokens should not be one line of SQL away from being readable.

I revoked those permissions outright and set the default so new tables don't acquire them, then verified by creating a table and confirming it arrived with none.

> SAY IT LIKE THIS
>
> *"There was no live vulnerability, but the protection came from an absence rather than a decision — nobody had written a policy yet. I made it explicit, because the failure mode was that a future reasonable-looking change would have quietly exposed the token table."*

Proving who the user is

Supabase Auth issues a JWT — a signed token the browser sends with every request. The backend verifies the signature before trusting anything in it.

It verifies against the provider's published keys (JWKS, asymmetric), not a shared secret. That matters practically: newer Supabase projects default to asymmetric keys, so a shared-secret implementation would

simply not work.

Three mistakes this project avoids

- **Trusting user-editable claims.** Part of a Supabase JWT can be edited by the user themselves, so it must never drive an authorisation decision. Nothing here uses it.
- **Hiding the wrong thing.** Asking for a mailbox that belongs to someone else returns "not found", not "forbidden" — "forbidden" would confirm the ID is real.
- **Trusting a client-supplied limit.** The emails endpoint caps its page size, because honouring `limit=1000000` hands anyone a way to make the server read the whole table.

9 How it's tested

SUITE	COUNT	WHAT IT COVERS
Backend (pytest)	381	Repositories against a real Postgres, services with the network faked, endpoints through the HTTP layer
Frontend (Vitest)	186	Component behaviour, loading and error states, API calls
Types + lint	–	TypeScript and ESLint, both clean

The tests had to be fixed before anything could be built

At the start, 132 backend tests were erroring — not failing, erroring. They opened a real connection to a Supabase project that had been deleted. A suite that needs a live remote database is a suite you cannot trust, so this was the first thing to fix.

The fix was to run Postgres locally via Supabase's CLI and to stop calling the remote sign-up API in test setup, inserting users directly into the users table instead.

> HOW I PROVED IT ACTUALLY WORKED
>
> I stopped the database container and re-ran the suite. It failed in exactly the same 132 places as before. That confirmed the tests were genuinely talking to the local database and not quietly passing for the wrong reason — a passing test suite can hide a test that no longer tests anything.

Test-driven, in a specific sense

For each new piece I wrote the tests first, watched them fail for the expected reason, then made them pass. The value isn't ceremony; it is that watching a test fail first proves the test can detect the thing it claims to detect.

This caught a real one. A timezone test passed when it should have failed, because my machine's timezone happened to make both branches agree. I rewrote it to derive a divergent instant regardless of machine timezone. Without checking the failure first, that test would have sat there asserting nothing.

Performance, measured rather than assumed

One dashboard query ran a subquery once per row. Rewriting it and adding indexes took it from **572 ms to 4 ms** — about 141× — and the query planner confirmed the inner loop count dropped from 1,000 to 1. Quoting the planner output rather than just the timing is the stronger version of this answer.

10 Bugs I found, and why each was invisible

"Tell me about a bug you fixed" is close to guaranteed. These are the strongest ones. Each shares a theme worth naming out loud: **they all failed silently.** Nothing crashed, no error appeared, and the system looked fine while doing the wrong thing.

---

1. Refreshing a token logged the user out of a working account

OAuth access tokens expire after about an hour, so the app exchanges a refresh token for a new one. That code always called Microsoft's library — written when Microsoft was the only provider. Once Google accounts existed, a Google refresh token was sent to Microsoft, failed, and the account was marked "needs reconnecting".

Why it was nasty: reconnecting produced the same Google token, which failed the same way. An unrecoverable loop, presented to the user as "your account expired" — pointing at the wrong cause entirely.

---

2. Linking a second mailbox corrupted the first

After signing in, the app reads which provider was used from the session's list of identities, taking the first match. Correct with one account. With two, the list order says nothing about which provider just issued the token — so linking Gmail to an existing Microsoft account stored the *Google* token under the *Microsoft* mailbox, breaking a working connection and never saving the new one.

The fix: the provider is now passed along on the redirect URL by whichever page started the flow — the only party that actually knows. The general lesson: don't infer something you can carry explicitly.

---

3. Refreshing tokens silently disabled your calendar

One mailbox is "primary" — the one calendar and Teams chat follow. The save function took a flag saying whether this account is primary, and a routine token refresh passes that flag as false, meaning "not asking". The code read it as "make this not primary". So every token refresh quietly stripped the primary flag, and the user lost calendar sync with nothing to explain why.

The fix was one line: treat false as "no opinion" rather than "set to false". Demotion now only happens when another account is explicitly promoted.

---

4. Two delete functions stopped deleting anything

When synced rows moved from being owned by a user to being owned by a mailbox, two delete functions were re-keyed from user ID to account ID. Both are UUIDs, so the call sites still compiled, still ran, and still reported success — while matching zero rows.

The fix: those parameters are now keyword-only, so a stale call fails loudly instead of succeeding emptily. Worth saying: the real lesson is that two values of the same type and different meaning is a design smell, and the type system should be made to care.

5. Deleting a mailbox left the user with no primary

Removing the primary mailbox deleted the row and stopped. The user still had working mailboxes but no primary, which silently disabled calendar and chat with nothing in the interface to explain it.

Now promotion happens in the same database transaction as the delete, so the two cannot come apart — and it prefers an Outlook mailbox over a Gmail one, because calendar and Teams are Microsoft-only features and promoting Gmail would leave them unsynced anyway.

---

6. A filter rule that was documented but never ran

Covered in section 7. The bulk-mail rule existed, was commented, and was unreachable — so every promotional email cost a full model call against the limit that binds first. This is a good one to mention because it is not a crash: it is a cost bug, found by reading rather than by failing.

> SAY IT LIKE THIS, IF ASKED FOR THE THEME
>
> *"The pattern across all of them is that nothing threw an error. A crash tells you where to look. These looked like success — a token refresh that returned fine, a delete that reported done, a filter rule that existed and ran zero times. That changed how I test: I now assert that the thing actually happened, not just that the call didn't fail."*

11 Trade-offs and what I chose not to build

Being able to name your own gaps, with reasons, is one of the clearest signals of engineering judgement. Volunteer these rather than waiting to be caught.

NOT BUILT	WHY, AND WHAT IT WOULD TAKE
Booking onto a Google calendar	Availability reads every connected calendar, but new meetings are created through Microsoft only, so they land on the Microsoft mailbox. Reading was the valuable half; writing needs a second implementation of event creation.
Extra Google calendars	Only the primary calendar of each Google account syncs — not "Family" or "Holidays". The sync bookmark is keyed by mailbox and resource name, and the name is free text, so this extends without a schema change.
Batching model calls	Deliberate. Measurements showed 4–10× headroom already, and batching makes it impossible to tell which message in a batch produced bad output.
Fetching Gmail headers before bodies	Would save some API quota, but the filter's length rule needs the body, and Gmail's quota is nowhere near binding. The tokens-per-day limit is, and the bulk filter is what addresses it.
Timezone-correct all-day events	Both providers' all-day events are treated as UTC midnight, so one can block the wrong slice of a day outside UTC. Known, documented, not yet fixed.

Things I would do differently

- **Versioned encryption keys from the start**, so the key can be rotated. Right now it cannot.
- **A real job queue instead of a cron hitting an HTTP endpoint.** It works and it is free, but it has no retries, no visibility, and no backpressure.
- **Integration tests against a recorded real response.** The provider clients are tested against fakes I wrote, which tests my understanding of the API rather than the API. Recording one real response per endpoint and replaying it would close that gap.

12 Likely questions, with answers

Why FastAPI and Postgres rather than just Next.js and Supabase?

"Supabase can expose tables directly, so for simple reads I wouldn't need a backend at all. But almost nothing here is a simple read — syncing from three APIs, running an AI pipeline, holding OAuth secrets, and enforcing a token budget all need a trusted server process. Once you need one, having the browser also read the database directly means authorisation lives in two places. So I kept one: the browser does login only, and every read goes through FastAPI."

How do you avoid re-downloading the whole mailbox every time?

"Each provider offers an incremental cursor, and I store one per mailbox per resource. The complication is that all three are different — Microsoft gives an opaque URL, Gmail an integer, Google Calendar an opaque token — and each expires differently. When a cursor expires I fall back to a thirty-day re-sync automatically rather than surfacing an error, because the alternative is an account that stays broken until someone notices."

What happens if the sync runs twice at the same time?

"It can't, now. A Postgres advisory lock keyed on the user ID means the second run reports 'already running' and does nothing. I used try-lock rather than wait-lock so cron workers don't stack up behind a slow sync. And I chose an advisory lock over a lock column because the database releases it automatically if the process dies — a column would stay stuck."

"Separately, every write is an upsert, so even if work were repeated it wouldn't duplicate rows. The lock is about not wasting the token budget."

How do you handle the language model returning garbage?

"Four layers. I request a strict JSON schema; I validate the reply against a Pydantic model before it reaches the database; I cap attempts at three so one unparseable message can't be retried forever and starve the queue; and I fence the message body in the prompt so an email can't issue instructions. A malformed reply is a handled outcome, not an exception."

This is a free tier. Does it actually scale?

"Not to many users, and I know exactly where it breaks. The quota is tracked per provider rather than per user, because the limit is organisation-wide — so one user exhausting it exhausts it for everyone. That's correct for a single-user tool and wrong for a product. The first thing I'd change is per-user budgeting with a paid key; the model and endpoint are already behind configuration, so that's an environment change, not a code change."

What was the hardest part?

"Changing the ownership model after the fact. Originally one Microsoft account per user — the token table's primary key *was* the user ID, so a second mailbox was impossible by construction. Supporting several meant adding an account reference to every synced table, changing what uniqueness means, and re-keying the sync bookmarks. The risk was that most of the ways to get it wrong are silent: a bookmark read from the wrong account just skips mail."

"I did it in one migration as nullable, then backfill, then not-null, and I verified by replaying all fifteen migrations from an empty database and running the full suite against the result."

How would you add a third provider, say Yahoo Mail?

"The shape is already there. Each provider has a client module that normalises its responses into one common dictionary, so everything downstream — the filter, the repositories, the UI — never learns which provider a message came from. Adding one means a new client module, a new branch in two dispatch points (sync and token refresh), and a provider value in the accounts table. The thing I'd be careful about is the incremental model, because all three existing ones differ."

What would you do with another month?

"First, prove the Google path against real accounts — it's tested but only against fakes. Second, replace the cron with a real job queue so failures retry and are visible. Third, versioned encryption keys, because right now the key can't be rotated. Those are in priority order: correctness, then operability, then the design weakness I'd be embarrassed by at scale."

13 Glossary

Every term used in this document. If you can define these, you can hold the conversation.

`OAuth`
A way to get permission to use someone's account without ever seeing their password. They approve on the provider's site; you receive a token.

`Access token`
Short-lived proof that you may read an account. Expires in about an hour.

`Refresh token`
Long-lived credential used to obtain new access tokens. The valuable one — losing it means reconnecting by hand.

`JWT`
A signed token carrying claims about the user. The signature proves it wasn't tampered with.

`JWKS`
The public keys a service publishes so others can verify its signatures without a shared secret.

`Scope`
How much access a token grants, e.g. read mail but not send it.

`REST API`
A server exposing operations as HTTP URLs.

`Endpoint`
One such URL, e.g. `/api/emails`.

`CORS`
The browser rule that a page may only call servers that have explicitly allowed its origin.

`Static site`
Pre-built files served as-is, with no server-side code at request time.

`Migration`
A numbered SQL file describing one schema change. Run in order, they rebuild the database exactly.

`Upsert`
Insert if absent, update if present. Makes repeating a write harmless.

`Idempotent`
Safe to do more than once with the same result. Upserts are; "add one" is not.

`Transaction`
A group of statements that all happen or none do. Prevents half-finished state.

`Foreign key`
A rule that a value must exist in another table. Stops orphaned rows.

`Cascade delete`
Deleting a row automatically deletes rows that depend on it.

`Index`
A lookup structure that turns scanning a table into jumping straight to the rows.

`Connection pool`
A reused set of database connections, because opening one per request is expensive.

`RLS`
Row Level Security — Postgres deciding which rows a role may see. With it on and no policy, nothing is visible.

`Advisory lock`
A named lock the database hands out for your own purposes. Released automatically when the session ends.

`Delta / incremental sync`
Fetching only what changed since a stored bookmark, rather than everything.

`Backfill`
Filling in historical data — here, the first thirty days of mail, or values for a newly added column.

`Token (LLM sense)`
A chunk of text the model counts, roughly ¾ of a word. Billing and limits are in these.

`Structured output`
Requiring the model to answer in a fixed schema rather than prose.

`Prompt injection`
Content trying to act as instructions. Fencing the content prevents it.

`Poison pill`

One permanently failing item retried forever, starving everything behind it. Capped attempts prevent it.

`Rate limit`

A cap on how much you may call an API per minute or per day.

`Cron`

A scheduler that runs something on a fixed interval.

`Cold start`

The delay when a free-tier server wakes from idle.

`Encryption at rest`

Data stored as ciphertext, so a stolen database dump isn't readable.

`Fernet`

The symmetric encryption scheme used here. One key both encrypts and decrypts.

Figures quoted throughout were measured on this project: 381 backend tests, 186 frontend tests, 15 migrations, 9 tables; truncation 3,900 → 700 tokens per long thread; one query 572 ms → 4 ms with the planner's inner-loop count dropping 1,000 → 1.